

STL库

基本概念：

1. 容器：可容纳各种数据类型的数据结构
2. 迭代器：可依次存取容器中元素的工具
3. 算法：用来操作容器中元素的函数模板

容器

分类

1. 顺序容器/序列容器 vector, deque, list
2. 关联容器/有序容器 set, multiset, map, multimap
3. 容器适配器 stack, queue, priority_queue

注：1/2统称为第一类容器

第一类容器特有的函数

- begin
- end
- rbegin
- rend
- erase
- clear

插入容器：

1. 插入的是**复制品**
2. 应该实现 == 和 < 运算符

顺序容器

1. vector 动态数组，尾端增删
2. deque 动态数组，两端增删
3. list 双向链表，不支持随机存取，任何位置增删

共同操作：

- front() 第一个元素的**引用**
- back() 最后一个元素的**引用**
- push_back() 末尾增加新元素
- pop_back() 删除容器末尾的元素

list容器

```
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

int main() {
    list<int> lst{1, 2, 3, 4, 5};

    lst.push_front(0);           // 头部插入
    lst.push_back(6);           // 尾部插入

    // 在值为3的位置前面插入 99
    auto it = find(lst.begin(), lst.end(), 3);
    if (it != lst.end()) {
        lst.insert(it, 99);
    }

    lst.remove(4);               // 删除所有值为4的元素（list特有，高效）

    for (int x : lst) {
        cout << x << " ";
    }
}
```

- `push_front()` / `pop_front()`：头部操作
- `erase()`：删除指定位置，**只有当前迭代器失效**，其他迭代器仍然有效
- `remove(val)`：删除所有等于 `val` 的元素
- `remove_if(pred)`：按条件删除
- `splice()`：把另一个 `list` 的节点**转移**过来
- `merge()`：合并两个**已排序**的 `list`
- `sort()`：`list` 自己的排序算法（不能用 `std::sort`，因为 `list` 不支持随机迭代器）
- `unique()`：删除**连续**的重复元素
- `reverse()`：反转链表

关联容器

1. set/multiset

集合 `set`不允许相同元素，`multiset`允许

2. map/multimap

映射 存放成对的key/value，根据key排序

map不允许相同key的元素，multimap允许

共有操作：

- `find(key)`：查找某个值，返回迭代器（找不到返回 `end()`）
- `lower_bound(key)`：返回**第一个不小于** `key` 的元素的迭代器 ($\geq$ `key`)
- `upper_bound(key)`：返回**第一个大于** `key` 的元素的迭代器 ($>$ `key`)
- `equal_range(key)`：同时返回 `lower_bound` 和 `upper_bound`，返回一个 `pair`
- `count(key)`：统计等于 `key` 的元素个数（`set / map` 只能是 0 或 1）
- `insert()`：插入元素

```
#include <iostream>
#include <set>
using namespace std;

int main() {
    set<int> s = {10, 20, 30, 30, 40}; // 30只会插入一次

    auto it = s.find(30);
    if (it != s.end()) {
        cout << "找到: " << *it << endl;
    }

    auto lb = s.lower_bound(25); // 指向 30
    auto ub = s.upper_bound(30); // 指向 40

    cout << "lower_bound(25) -> " << *lb << endl;
    cout << "upper_bound(30) -> " << *ub << endl;

    return 0;
}
```

容器适配器

1. `stack` 栈 后进先出
2. `queue` 队列 先进先出
3. `priority_queue` 优先级队列，优先级最高的总是第一个出列

迭代器

指向**第一类容器**中的元素

定义

容器类名::iterator 变量名 或

容器类名::const iterator 变量名

访问指向的元素: * 迭代器变量名

指向下一个元素: ++

past-the-end值: 容器中最后一个元素的后面, 用来访问对象是非法的

分类——由弱到强

所有: p++, ++p

1. 输入 只读 *p, p=p1, p==p1, p!=p1
2. 输出 只写 *p, p=p1
3. 正向 读写 以上全部
4. 双向 读写+向前向后 以上全部, p--, --p list/set/multiset/map/multimap
5. 随机访问 读写+随机移动 以上全部, 以及p+=i, p-=i, p+i, p-i; p<p1, p<=p1, p>p1, p>=p1, p[i]
vector/deque

算法

通过迭代器来操纵容器中的元素的函数模板

分类

- 不变序列 不会修改所作用的容器或对象
- 变值 会修改原区间或目标区间, **不可以属于关联容器**
- 删除 不会使容器里的元素减少, 留下元素前移, **没填上的维持原来的值**
- 变序 改变容器中元素的顺序, **不适用于关联容器**
- 排序 需要支持随机访问迭代器
- 有序区间 已经排好序+随机访问迭代器
- 数值

STL 中的大小相等判断

1. 缺省的情况下, 比较大小是用 < 进行的, **与>无关**
2. 相等判断
 1. 在**未排序**的区间用 ==, 例如find
 2. 在**排好序**的区间
相等等价于**x<y与y<x同时成立**
与 == 运算符无关

find

```
#include <algorithm>

std::find(begin, end, value);
```

参数：

- `begin`：起始迭代器
- `end`：结束迭代器
- `value`：要查找的值

返回值：

- 找到 → 返回指向该元素的迭代器
- 未找到 → 返回 `end`

```
#include <iostream>
#include <vector>
#include <algorithm>

int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    auto it = std::find(v.begin(), v.end(), 3);

    if (it != v.end()) {
        std::cout << "找到元素： " << *it << std::endl;
    } else {
        std::cout << "未找到元素" << std::endl;
    }

    return 0;
}
```

copy

```
#include <algorithm>

std::copy(begin, end, dest_begin);
```

参数说明：

- `begin`：源区间起始迭代器
- `end`：源区间结束迭代器
- `dest_begin`：目标区间起始迭代器

返回值：返回指向目标区间**最后一个被拷贝元素的下一个位置**的迭代器

```
#include <iostream>
#include <vector>
```

```
#include <algorithm>

using namespace std;

int main() {
    vector<int> v1 = {1, 2, 3, 4, 5};
    vector<int> v2(5);    // 必须提前开空间!

    copy(v1.begin(), v1.end(), v2.begin());

    for (int x : v2)
        cout << x << " ";
}
```